

PROGRAMMAZIONE II

ORIENTAZIONE AGLI OGGETTI E UML

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

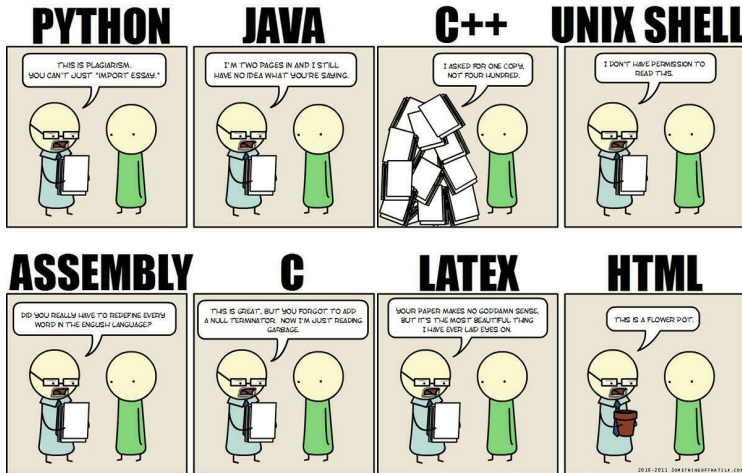
A.A. 2025/2026

LINGUAGGI DI PROGRAMMAZIONE



Non esiste un linguaggio di programmazione universale (anche se qualcuno ci ha provato alla fine degli anni '70)

LO ZOO DEI LINGUAGGI DI PROGRAMMAZIONE (CONT.)



PERCHÉ ABBIAMO BISOGNO DI DIVERSI LINGUAGGI DI PROGRAMMAZIONE?

«L'informatica non riguarda i computer più di quanto l'astronomia riguardi i telescopi.»

E.W. Dijkstra

Se tutto ciò che hai è un martello, ogni cosa sembra un chiodo



PERCHÉ ABBIAMO BISOGNO DI DIVERSI LINGUAGGI DI PROGRAMMAZIONE? (CONT.)

I programmatori moderni hanno bisogno di **astrazione**

Negli anni '50/'60 l'obiettivo era l'efficienza del codice macchina

- ▶ Collegamento diretto tra il linguaggio e l'**implementazione hardware**
- ▶ Programmatori economici, macchine costose: *tenere occupata la macchina!*

Oggi l'obiettivo è l'efficienza del codice del programma

- ▶ Collegamento diretto tra il linguaggio e la **progettazione del software**
- ▶ Programmatori costosi, macchine economiche: *tenere occupato il programmatore!*

PARADIGMI DI PROGRAMMAZIONE

Linguaggi imperativi/procedurali

- ▶ Basati sulla nozione di stato (locazioni di memoria contenenti valori)
- ▶ Le istruzioni sono comandi che modificano lo stato (es. assegnamenti)
- ▶ Codice organizzato in blocchi logici per il riuso
- ▶ C, Pascal, FORTRAN, COBOL, . . .

Linguaggi dichiarativi

- ▶ Basati sulla nozione di funzione o relazione
- ▶ Le istruzioni sono dichiarazioni di nuovi valori
- ▶ Lisp, Haskell, Prolog, SQL, . . .

PARADIGMI DI PROGRAMMAZIONE (CONT.)

Linguaggi funzionali (dichiarativi)

- ▶ Il risultato di un programma è il valore esplicito di un'espressione
- ▶ Definizione e applicazione di funzioni, ricorsione
- ▶ Programmare si riduce a definire una funzione che calcola un risultato

```
// Haskell 1
add :: Integer -> Integer -> Integer // dichiarazione di funzione 2
add x y = x + y // definizione di funzione 3

main = do 4
  putStrLn "The addition of the two numbers is:" 5
  print(add 2 5) // chiamata di funzione 6 7
```

PARADIGMI DI PROGRAMMAZIONE (CONT.)

Linguaggi logici (dichiarativi)

- ▶ Il risultato del programma è il valore di alcune variabili che soddisfano relazioni
- ▶ Implicazioni logiche tra formule, regole di riscrittura
- ▶ Programmare si riduce a definire le relazioni tra variabili

```
// Prolog
happy(sam).           // fatto 1
listens2Music(mia).   // fatto 2
listens2Music(sam):- happy(sam).           // regola 1
playsAirGuitar(mia):- listens2Music(mia).   // regola 2
playsAirGuitar(sam):- listens2Music(sam).   // regola 3
```

Query: ?- playsAirGuitar(mia), playsAirGuitar(sam).

// Risposta: sì

PARADIGMI DI PROGRAMMAZIONE (CONT.)

Linguaggi orientati agli oggetti

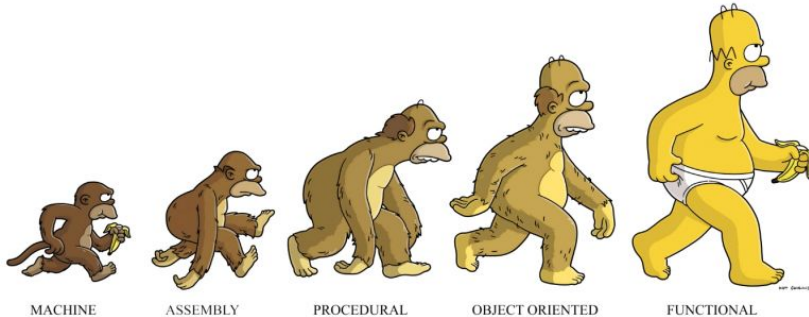
- ▶ Linguaggio imperativo con qualche iniezione di concetti dichiarativi
- ▶ Gli oggetti contengono dati (tratto imperativo)
- ▶ Gli oggetti contengono metodi che operano sui dati (tratto dichiarativo)
- ▶ Basati sul passaggio di messaggi
- ▶ Astrazioni: incapsulamento, ereditarietà, . . .
- ▶ Smalltalk, Java, C++

Tendenza a sviluppare linguaggi multi-paradigma (non sempre un bene)

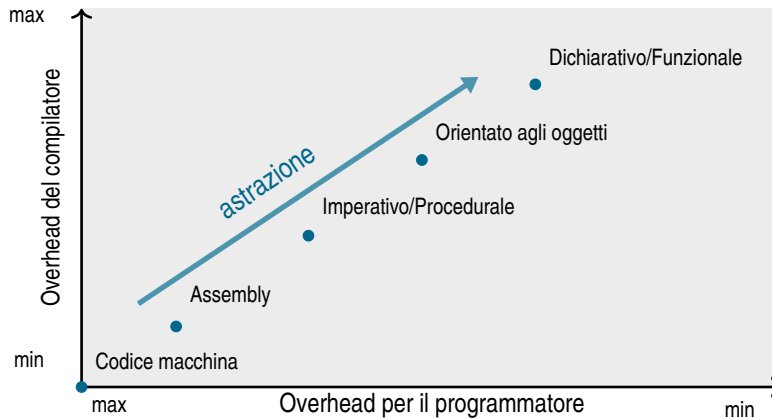
BREVE STORIA



VERSO L'ASTRAZIONE



VERSO L'ASTRAZIONE (CONT.)



ORIENTAZIONE AGLI OGGETTI



LINGUAGGI PROCEDURALI

- ▶ Raggruppano le istruzioni in blocchi di codice
- ▶ Dati: vect[20]
- ▶ Funzioni: sort()
- ▶ Relazioni implicite di lettura e scrittura tra dati e funzioni

```
// Programma C per gestire vettori di interi
```

```
int vect[20];

void sort() { /* sort */ }
int search(int n){ /* search */ }
void init() { /* init */ }

int i;

void main() {
    init();
    sort();
    search(13);
}
```

LINGUAGGI PROCEDURALI

- ▶ Nessuna relazione esplicita tra vettori e operazioni sui vettori
- ▶ Nessun controllo dei limiti del vettore
 - ▶ for (i=0; i <= 21; i++) vect[i]=0;
- ▶ Nessun controllo sull'inizializzazione del vettore
 - ▶ init() viene eseguita?

// Programma C per gestire vettori di interi

```
int vect[20];

void sort() { /* sort */ }
int search(int n){ /* search */ }
void init() { /* init */ }

int i;

void main() {
    init();
    sort();
    search(13);
}
```

In generale, il vettore non può essere un concetto autonomo e modulare!

LIMITAZIONI DEI LINGUAGGI PROCEDURALI

Il programmatore è responsabile dell'integrità e riservatezza dei dati

- ▶ Nessun vincolo sulle relazioni di lettura e scrittura tra dati
- ▶ Funzioni esterne possono influenzare i dati del vettore

Sul lungo termine

- ▶ Ogni funzione può leggere e scrivere ogni dato (negativo)
- ▶ Troppe relazioni implicite da controllare manualmente
- ▶ Il codice diventa difficile da comprendere e mantenere (spaghetti code)

GLI OGGETTI IN SOCCORSO

Linguaggi procedurali

- ▶ Il programma definisce i dati e poi chiama sottoprogrammi che agiscono sui dati

Linguaggi orientati agli oggetti

- ▶ Il programma crea oggetti che incapsulano sia i dati che le operazioni sui dati

L'orientazione agli oggetti è semplicemente un nuovo modo di organizzare i programmi

- ▶ Non può fare nulla che non possa essere fatto usando un linguaggio procedurale

VANTAGGI DEI LINGUAGGI ORIENTATI AGLI OGGETTI

Linguaggi orientati agli oggetti

- ▶ Consentono ai programmatori di usare blocchi di codice senza conoscere l'intero quadro
- ▶ Rendono il riuso del codice una possibilità reale
- ▶ Semplificano la manutenzione e l'evoluzione

Questi sono punti critici quando

- ▶ Si considera una base di codice di grandi dimensioni
- ▶ Molte persone e team lavorano insieme

VANTAGGI DEI LINGUAGGI ORIENTATI AGLI OGGETTI (CONT.)

Programmi con meno di 30 LoC

- ▶ Lo spaghetti code è comprensibile e più rapido da scrivere
- ▶ Lo spaghetti code è ragionevolmente facile da mantenere

Programmi con più di 1000 LoC

- ▶ Lo spaghetti code è incomprensibile
- ▶ Lo spaghetti code probabilmente non funziona
- ▶ Lo spaghetti code è molto difficile da mantenere

Solo i programmi con più di 1000 LoC traggono reale beneficio dall'orientazione agli oggetti

PROGETTAZIONE ORIENTATA AGLI OGGETTI

Dato un sistema, con componenti e relazioni tra di essi

- ▶ Identificare i componenti
- ▶ Definire l'interfaccia dei componenti
- ▶ Definire come i componenti interagiscono tra loro tramite la loro interfaccia
- ▶ Minimizzare le relazioni tra i componenti

I programmatori sono forzati in questa mentalità dal linguaggio

PROGETTAZIONE ORIENTATA AGLI OGGETTI (CONT.)

Definisce un nuovo tipo di componente

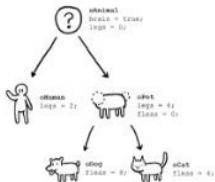
- ▶ Oggetto (e classe)
- ▶ Sia i dati che le operazioni sui dati sono all'interno dello stesso modulo
- ▶ Permette di definire un'interfaccia più precisa

Definisce un nuovo tipo di relazione

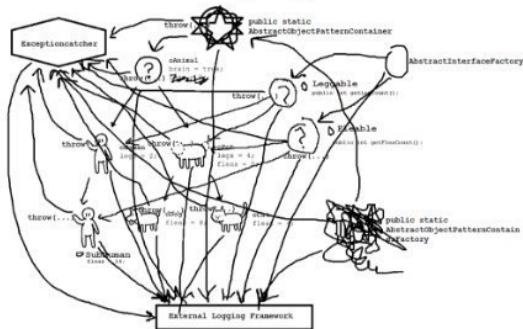
- ▶ Scambio di messaggi tra oggetti
- ▶ Le operazioni di lettura e scrittura sono limitate all'ambito dell'oggetto

PROGETTAZIONE ORIENTATA AGLI OGGETTI (CONT.)

What OOP users claim



What actually happens



UNIFIED MODELING LANGUAGE



UML



Unified Modeling Language

- ▶ Linguaggio standard di modellazione e specifica, ampiamente adottato
- ▶ Per definire e documentare graficamente un sistema orientato agli oggetti

DIAGRAMMA DELLE CLASSI

UML supporta molti tipi di diagrammi (attività, casi d'uso, sequenza)

Siamo interessati ai diagrammi delle classi

- ▶ Catturare concetti astratti // es. automobili
- ▶ Modellare i dati associati ai concetti // es. produttore dell'auto
- ▶ Modellare le relazioni tra i concetti // es. automobili e camion sono veicoli
- ▶ Modellare il comportamento del concetto // es. le automobili non possono volare

CLASSE

Una classe rappresenta un insieme di oggetti (es. un tipo)

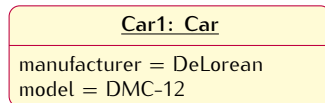
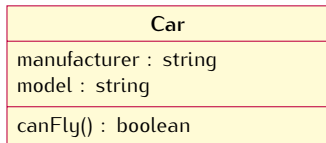
- ▶ Aveni proprietà comuni
- ▶ Aveni esistenza autonoma
- ▶ Un'istanza di una classe è un **oggetto** del tipo che la classe rappresenta

Car
manufacturer : string model : string
canFly() : boolean

OGGETTO

Un oggetto rappresenta un elemento fisico o logico

- ▶ Ha identità e attributi
- ▶ Ha operazioni che può compiere (comportamento)
- ▶ Ha messaggi che può inviare e ricevere



CLASSI E OGGETTI

Una classe

- ▶ È la descrizione della struttura dell'oggetto
- ▶ Definisce i dati, chiamati attributi o campi
- ▶ Definisce le operazioni sui dati, chiamate metodi
- ▶ Definisce le procedure di creazione dell'oggetto, chiamate costruttori

Un oggetto

- ▶ È un'istanza di una classe
- ▶ Contiene valori di dati reali (stato)
- ▶ Ha un'identità, per distinguere oggetti della stessa classe

CLASSI E OGGETTI (CONT.)

Una classe è una definizione di tipo

- ▶ Di norma non viene allocata memoria finché non viene creato un oggetto dalla classe

La creazione di un oggetto è chiamata istanziamento

- ▶ L'oggetto creato è spesso chiamato un'istanza della classe

Non c'è limite al numero di oggetti che possono essere creati da una classe

- ▶ . . . heap permettendo

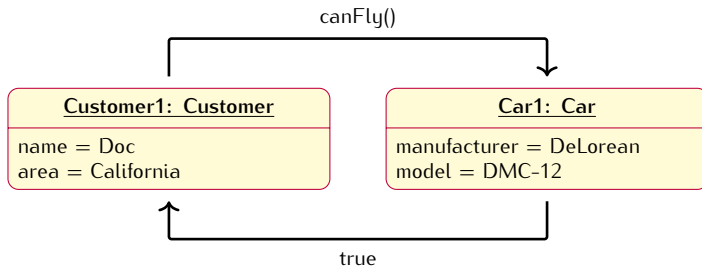
Ogni oggetto è indipendente

- ▶ Interagire con un oggetto non influenza gli altri

COME INTERAGISCONO GLI OGGETTI

Gli oggetti comunicano tramite passaggio di messaggi

- ▶ Non mediante chiamata di procedura o accesso diretto ai dati locali degli oggetti
- ▶ Un messaggio è una richiesta/risposta di servizio



INTERFACCIA DELL'OGGETTO

L'interfaccia di un oggetto è l'insieme dei messaggi che può ricevere

- ▶ Qualsiasi altro messaggio è illegale
- ▶ Il messaggio è mappato su una funzione all'interno dell'oggetto
- ▶ Semplicemente il sottoinsieme di metodi che altri oggetti possono chiamare

Car
<i>public</i> manufacturer : string <i>private</i> model : string
<i>public</i> canFly() : boolean <i>private</i> setFrameNumber(uint num)

Visibilità di campi e metodi:

public significa accessibile da ovunque

private significa accessibile solo all'interno della classe

INCAPSULAMENTO E INFORMATION HIDING

Incapsulamento: raggruppare dati e operazioni sui dati all'interno di un'unica unità

Information hiding: limitare l'accesso esterno a determinate parti dell'unità

Vantaggi

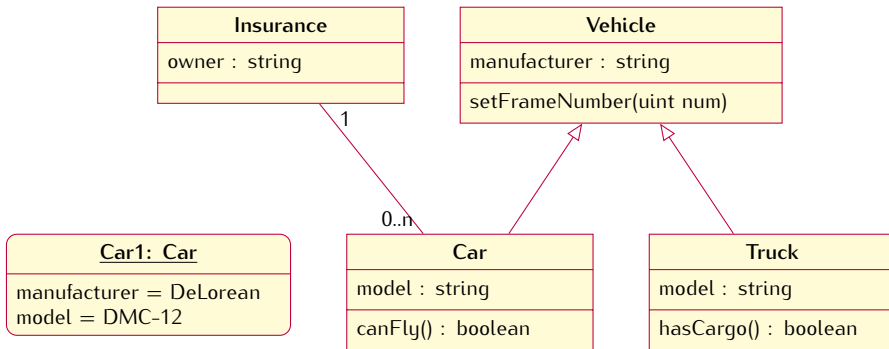
- ▶ Per usare un oggetto, è solo necessario comprenderne l'interfaccia
- ▶ L'implementazione dell'oggetto non necessita di codice esterno
- ▶ L'implementazione dell'oggetto può cambiare senza richiedere il refactoring di altro codice
- ▶ Le modifiche al codice avvengono in un'unica posizione (meno errori)

ASSOCIAZIONI TRA CLASSI

Relazioni logiche tra classi

// ne vedremo altre più avanti nel corso

► Ereditarietà



I PILASTRI DELL'ORIENTAZIONE AGLI OGGETTI

Encapsulation



Polymorphism



Inheritance



Abstraction



DOMANDE E RISPOSTE

